Name utsav sabharwal

Roll NO. : 2401010079

Section-B

Course : B .Tech CSE Core

Assignment: 1

❖ **Aim :**
To build a **dynamic To-Do List web application** using **HTML, CSS, JavaScript**, and a **database backed** to **add, edit, delete, and manage tasks permanently** with full CRUD functionality.

❖ **Tasks:**

- **Project Setup & Introduction** – Create project folders (frontend & backend) and link HTML, CSS, JS files.
- **UI Layout (HTML)** – Input field for task, optional description, priority dropdown, Add Task button, task list section, filter buttons (All/Active/Completed), and counters.
- **Styling (CSS)** – Centered , mobile-friendly layout, completed task styling, priority badges, button hover effects.
- **Frontend Interactivity (JS)** – Add, edit, delete, complete tasks; filter tasks; show counts; input validation.
- **Backend + Database** – Use Node.js + Express + SQLite (or other backend) to persist tasks.
- **REST API Implementation** – CRUD endpoints: GET, POST, PUT, PATCH, DELETE for tasks.
- **Bonus Features (Optional)** – Search, sort, due dates, local Storage fallback, JSON export, basic authentication.

❖ **Tech Stack Used:**
- **Frontend:** HTML, CSS, JavaScript
- **Backend:** Node.js, Express
- **Database:** SQLite (or MySQL/other as per setup)
- **APIs:** RESTful APIs for CRUD operations

❖ Html Code :

```
<!doctype html>
<html lang="en">

<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />

  <title>My To-Do List</title>

  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>

  <link
href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&displa
y=swap" rel="stylesheet">

  <link rel="stylesheet" href="style.css">
</head>

<body>

  <main class="app-shell">

    <section class="todo-app">

      <!-- HERO -->
```

```
    <div class="hero">
      <h1>My To-Do List</h1>
      <p class="subtitle">
      </p>
    </div>

    <!-- FORM -->
    <section class="panel form-panel">

      <form id="taskForm" class="task-form">

        <div class="field-group title-group">
          <label for="taskTitle">Task title</label>

          <input              id="taskTitle"
name="title"              type="text"
maxlength="50"
placeholder="Enter task title"
required
          >

          <small class="hint">
            Required · maximum 50 characters
          </small>
        </div>
        <div class="field-group">
```

```
          <label for="taskDescription">Description (optional)</label>

          <textarea              id="taskDescription"
name="description"              rows="3"
placeholder="Add more details about the task">
          </textarea>
        </div>

        <div class="field-row">

          <div class="field-group compact-field">
            <label for="taskPriority">Priority</label>

            <select id="taskPriority" name="priority">
              <option value="Low">Low</option>
              <option value="Medium" selected>Medium</option>
              <option value="High">High</option>
            </select>
          </div>

         <div class="field-group compact-field">
           <label for="sortTasks">Sort by</label>
            <select id="sortTasks" name="sortTasks">
              <option value="createdAt-desc">Newest first</option>
              <option value="createdAt-asc">Oldest first</option>
              <option value="priority-asc">Priority: High to Low</option>
```

```
            <option value="priority-desc">Priority: Low to High</option>
            <option value="title-asc">Title A-Z</option>
          </select>
        </div>

      </div>

      <div class="form-actions">

        <button type="submit" id="submitButton" class="primary-btn">
          Add Task
        </button>

        <button              type="button"
id="cancelEditButton"
class="secondary-btn hidden">
          Cancel Edit
        </button>

      </div>

    </form>
  </section>

  <!-- TOOLBAR -->
  <section class="panel toolbar-panel">
```

```
<div class="search-box">
  <label class="sr-only" for="searchTasks">Search tasks</label>

  <input            id="searchTasks"
type="search"            placeholder="Search by title
or description"
  >
</div>

<div class="filter-group" role="group" aria-label="Filter tasks">

  <button type="button" class="filter-btn active" data-filter="all">
    All
  </button>

  <button type="button" class="filter-btn" data-filter="active">
   Active
  </button>

  <button type="button" class="filter-btn" data-filter="completed">
                                                                Completed
  </button>

</div>

</section>
```

```
<!-- STATS -->
<section class="panel stats-panel">

  <div>
    <span class="stat-label">Completed / Total</span>
    <strong id="taskCounter">0 / 0</strong>
  </div>

  <div>
    <span class="stat-label">Active tasks</span>
    <strong id="activeCounter">0</strong>
  </div>

  <button type="button" id="exportButton" class="secondary-btn">
    Export JSON
  </button>

</section>

<!-- TASK LIST -->
<section class="panel list-panel">

  <div id="statusMessage" class="status-message" aria-live="polite"></div>

  <ul id="taskList" class="task-list"></ul>
```

```
        <div id="emptyState" class="empty-state">
          <h2>No tasks found</h2>
          <p>Add your first task or adjust the filters/search.</p>
</div>

      </section>

    </section>

  </main>

  <!-- TASK TEMPLATE -->
  <template id="taskTemplate">

    <li class="task-item">

      <label class="checkbox-wrap">
       <input type="checkbox" class="task-checkbox">
       <span class="checkbox-ui"></span>
     </label>
      <div class="task-content">

        <div class="task-header-row">
          <h3 class="task-title"></h3>
          <span class="priority-badge"></span>
```

```
      </div>

      <p class="task-description"></p>

      <span class="task-date"></span>

    </div>

    <div class="task-actions">

      <button type="button" class="icon-btn edit-btn">
        Edit
      </button>

      <button type="button" class="icon-btn delete-btn">
        Delete
      </button>

    </div>

  </li>
```

```
  </template>

  <script src="script.js"></script>

</body>
</html>
```

❖ CSS Code :

```
❖ :root{
❖ --primary:#6366f1;
❖ --primary-dark:#4f46e5;
❖ --success:#22c55e;
❖ --warning:#f59e0b;
❖ --danger:#ef4444;
❖ --text:#ffffff;
❖ --muted:#cbd5f5;
❖ --radius:14px;
❖ }
❖
❖ /* RESET */
❖
❖ *{
❖ box-sizing:border-box;
❖ margin:0;
```

```
❖
❖
❖
❖
```

```
padding:0;
}

/* BACKGROUND */

body{
font-family:"Inter",sans-serif;

background:
linear-gradient(rgba(0,0,0,0.45),rgba(0,0,0,0.45)),
url("https://s3.amazonaws.com/eap04.easyagentpro.com-live/wpcontent/uploads/sites/165/2023/10/27125126/Discipline.jpg");
```

```
background-size:cover;
background-position:center;
background-repeat:no-repeat;
background-attachment:fixed;

min-height:100vh;
color:white;
}
```

```
/* APP CONTAINER */

.app-shell{
max-width:1100px;
margin:auto;
padding:40px 20px;
}

.todo-app{
display:grid;
gap:22px;
}

/* HERO */

.hero{
text-align:center;
}

.hero h1{
font-size:2.8rem;
font-weight:700;
}
```

```
❖
❖
❖
❖
❖
❖
❖
❖     .subtitle{
❖     color:#e2e8f0;
❖     }
❖
❖     /* GLASS PANELS */
❖
❖     .panel{
      background:rgba(255,255,255,0.08);
backdrop-filter:blur(12px);      border:1px
solid rgba(255,255,255,0.2);      border-
radius:var(--radius);      padding:22px;
box-shadow:0 8px 30px rgba(0,0,0,0.3);
color:white;
}

 /* FORM */

 .task-form{
display:grid;
gap:16px;
❖     }
❖
❖     .field-group{
❖     display:grid;
❖     gap:6px;
❖     }
❖
❖
❖
❖
❖
❖
❖
❖
```

```
input,textarea{
background:rgba(255,255,255,0.15);
border:1px solid rgba(255,255,255,0.3);
border-radius:10px;
padding:12px;
color:white;
font-size:14px;
}

input::placeholder,
textarea::placeholder{
color:#e2e8f0;
}

input:focus,
textarea:focus{
outline:none;
border-color:var(--primary);
box-shadow:0 0 0 2px rgba(99,102,241,0.4);
}

/* SELECT DROPDOWN */
```

```
select{
background:rgba(255,255,255,0.15);
border:1px solid rgba(255,255,255,0.3);
border-radius:10px;      padding:12px;
color:black;
}

/* DROPDOWN OPTIONS */

option{
color:black;
background:white;
}
```

```
/* BUTTONS */
.primary-btn{
background:linear-gradient(135deg,#6366f1,#8b5cf6);
border:none;
padding:12px 20px;
border-radius:10px;
color:white;
font-weight:600;
cursor:pointer;
transition:0.25s;
}

.primary-btn:hover{
transform:translateY(-2px);
box-shadow:0 6px 18px rgba(99,102,241,0.5);
}

.secondary-btn{
background:rgba(255,255,255,0.2);
border:none;
padding:12px 20px;
border-radius:10px;
color:white;
```

```
    cursor:pointer;
    transition:0.2s;
    }

    .secondary-btn:hover{
```

```
❖
❖
❖
❖
❖
❖
❖
    background:rgba(255,255,255,0.35);
}

/* TOOLBAR */

.toolbar-panel{
    display:flex;
    justify-
    content:space-
    between;
    align-items:center;
gap:14px;      flex-
wrap:wrap;
}

.search-box input{
❖    width:260px;
❖    }
❖
❖    /* FILTER */
❖
❖    .filter-group{
❖    display:flex;
❖    gap:8px;
❖    }
```

```
❖
❖
❖
❖
❖
❖
❖
```

```
.filter-btn{
background:rgba(255,255,255,0.15);
border:none;
padding:8px 16px;
border-radius:20px;
color:white;
cursor:pointer;
}

.filter-btn.active{
background:var(--primary);
}

/* STATS */

.stats-panel{
display:flex;
justify-content:space-between;
align-items:center;
flex-wrap:wrap;
}
```

```
.stat-label{      font-
size:12px;
color:#cbd5f5;
}
/* TASK LIST */

.task-list{      list-
style:none;
display:grid;
gap:12px;
}
```

```
/* TASK ITEM */

.task-item{
display:grid;
grid-template-columns:auto 1fr auto;
gap:16px;
padding:16px;
border-radius:12px;
background:rgba(255,255,255,0.12);
border:1px solid rgba(255,255,255,0.25);
backdrop-filter:blur(10px);
transition:0.2s;
}
```

```
.task-item:hover{
transform:translateY(-3px);
box-shadow:0 6px 18px rgba(0,0,0,0.4);
}

/* CHECKBOX */

.checkbox-ui{
width:20px;
height:20px;
border-radius:50%;
border:2px solid white;
display:inline-block;
}

.task-checkbox{
display:none;
}

.task-checkbox:checked + .checkbox-ui{
background:var(--success);  border-color:var(--success);
}

/* TASK CONTENT */
```

```
❖
❖
❖
❖
❖
❖
❖
 .task-title{      font-
size:16px;      font-
weight:600;
```

```
❖
❖
❖
❖
❖
❖
❖
```

```
}

.task-description{ ❖     margin:6px 0;
color:#e2e8f0;
}

.task-date{
font-size:12px;
color:#cbd5f5;
}

.task-item.completed .task-title{
text-decoration:line-through;
opacity:0.6;
}

/* PRIORITY */

.priority-badge{
font-size:12px;
font-weight:600;
padding:4px 10px;
border-radius:20px;
}
```

```
.priority-low{
background:#22c55e;
}
```

```
.priority-medium{
background:#f59e0b;
}

.priority-high{
background:#ef4444;
}

/* ACTION BUTTONS */

.task-actions{
display:flex;
gap:6px;
}

.icon-btn{
border:none;
background:rgba(255,255,255,0.2);
padding:6px 10px;
border-radius:8px;
color:white;
cursor:pointer;
}

.icon-btn:hover{
```

```
background:rgba(255,255,255,0.35);
}

/* EMPTY */

.empty-state{
text-align:center;
color:#cbd5f5;
padding:30px;
}

/* MOBILE */

@media(max-width:700px){

.task-item{
grid-template-columns:auto 1fr;
}

.task-actions{ grid-column:1/-1;
}
```

```
❖
❖
❖
❖
❖
❖
❖
.toolbar-panel{ flex-direction:column; align-items:flex-start;
```

```
❖
❖
❖
❖
❖
❖
❖
```

❖
❖
❖

```
}

}
```

❖ JS Code :

❖ Script.js :

```
❖     const TITLE_LIMIT = 50;
❖
❖     const taskForm = document.getElementById("taskForm");
❖     const taskTitle = document.getElementById("taskTitle");
❖     const taskDescription = document.getElementById("taskDescription");
❖     const taskPriority = document.getElementById("taskPriority"); ❖
❖     const taskList = document.getElementById("taskList");
❖     const emptyState = document.getElementById("emptyState"); ❖
❖     const taskCounter = document.getElementById("taskCounter");
❖     const activeCounter = document.getElementById("activeCounter"); ❖
❖     const taskTemplate = document.getElementById("taskTemplate"); ❖
❖     let tasks = [];
❖
❖     /* LOAD TASKS */
❖
❖     document.addEventListener("DOMContentLoaded", () => { ❖
❖     const savedTasks = localStorage.getItem("tasks"); ❖
❖     if(savedTasks){
❖     tasks = JSON.parse(savedTasks);
❖     }
❖
❖     renderTasks();
❖     });
❖
❖     /* ADD TASK */
❖
❖     taskForm.addEventListener("submit", function(e){ ❖
❖     e.preventDefault();
❖
❖     const title = taskTitle.value.trim();
```

```
❖    const description = taskDescription.value.trim();
❖    const priority = taskPriority.value; ❖
❖    if(!title){
❖    alert("Task title required");
❖    return;
❖    }
❖
❖    if(title.length > TITLE_LIMIT){
❖    alert("Title must be less than 50 characters");
❖    return;
❖    }
❖
❖    const task = {
❖    id: Date.now(),
❖    title,
❖    description,
❖    priority,
❖    isDone:false,
❖    createdAt:new Date()
❖    };
❖
❖    tasks.unshift(task);
❖
❖    saveTasks();
❖
❖    renderTasks();
❖
❖    taskForm.reset();
❖    });
❖
❖    /* RENDER TASKS */ ❖
❖    function renderTasks(){
❖
❖    taskList.innerHTML="";
❖
❖    tasks.forEach(task=>{
❖
❖    const fragment = taskTemplate.content.cloneNode(true); ❖
```

```
❖     const taskItem = fragment.querySelector(".task-item");
❖     const checkbox = fragment.querySelector(".task-checkbox");
❖     const title = fragment.querySelector(".task-title");
```

```
❖     const description = fragment.querySelector(".task-description");
❖     const date = fragment.querySelector(".task-date");
❖     const badge = fragment.querySelector(".priority-badge");
❖     const deleteBtn = fragment.querySelector(".delete-btn"); ❖
❖     title.textContent = task.title;
❖     description.textContent = task.description || "No description"; ❖
❖     date.textContent = new Date(task.createdAt).toLocaleString(); ❖
❖     badge.textContent = task.priority;
❖
❖     badge.classList.add(`priority-${task.priority.toLowerCase()}`); ❖
❖     checkbox.checked = task.isDone; ❖
❖     taskItem.classList.toggle("completed",task.isDone); ❖
❖     checkbox.addEventListener("change",()=>{ ❖
❖     task.isDone = checkbox.checked; ❖
❖     saveTasks();
❖
❖     renderTasks();
❖     });
❖
❖     deleteBtn.addEventListener("click",()=>{ ❖
❖     tasks = tasks.filter(t=>t.id!==task.id); ❖
❖     saveTasks(); ❖
❖     renderTasks();
❖     });
❖
❖     taskList.appendChild(fragment);
❖     }); ❖
❖     emptyState.style.display = tasks.length ? "none":"block"; ❖
❖     updateCounters();
❖     }
❖
```

```
❖     /* SAVE TASKS */ ❖
❖     function saveTasks(){
❖
❖     localStorage.setItem("tasks",JSON.stringify(tasks));
❖ }
❖
❖     /* UPDATE COUNTERS */
❖
❖     function updateCounters(){
❖
❖     const total = tasks.length;
❖
❖     const completed = tasks.filter(t=>t.isDone).length;
❖
❖     taskCounter.textContent = `${completed} / ${total}`;
❖
❖     activeCounter.textContent = total-completed; ❖ }
```

❖ Package.json:

{ "name": "dynamic-todo-list-backend", "version": "1.0.0", "description": "Backend API for the Dynamic To-Do List assignment", "main": "server.js", "scripts": {

"start": "node server.js", "dev": "node server.js"

},"keywords": ["todo", "express", "sqlite"

],"license": "ISC", "dependencies": { "cors": "^2.8.6", "express": "^4.22.1", "sqlite3": "^5.1.7"

},"author": "", "type": "commonjs"

}

❖Server.js:

```
const express = require("express");
const cors = require("cors"); const
path = require("path");
const sqlite3 = require("sqlite3").verbose();

const app = express(); const PORT =
process.env.PORT || 4000;

/* DATABASE */

const dbPath = path.join(__dirname, "db.sqlite"); const
db = new sqlite3.Database(dbPath);

/* FRONTEND */

const frontendPath = path.join(__dirname, "..", "frontend");

/* MIDDLEWARE */

app.use(cors()); app.use(express.json());
app.use(express.static(frontendPath));

/* CREATE TABLE */

db.serialize(() => {
db.run(`
```

```
    CREATE TABLE IF NOT EXISTS tasks (      id
 INTEGER PRIMARY KEY AUTOINCREMENT,
 title TEXT NOT NULL,
     description TEXT DEFAULT '',      priority TEXT
DEFAULT 'Medium',      isDone INTEGER DEFAULT 0,
createdAt DATETIME DEFAULT CURRENT_TIMESTAMP
    )
  `);
});

/* HELPERS */

function normalizeTask(row) {
return {
    ...row,
    isDone: Boolean(row.isDone),
  };
}

function validateTitle(title) {
return (     typeof title ===
"string" &&
title.trim().length > 0 &&
title.trim().length <= 50
  );
}
```

```
function validatePriority(priority) { return ["Low",
  "Medium", "High"].includes(priority);
}

/* GET ALL TASKS */

app.get("/tasks", (req, res) => {

  db.all(
    "SELECT id,title,description,priority,isDone,createdAt FROM tasks",
    [],
    (error, rows) => {
      if (error) {         return res.status(500).json({ message: "Failed to
fetch tasks." });
      }

      const tasks = rows.map(normalizeTask);

      res.json(tasks);
    }
  );
});

/* CREATE TASK */

app.post("/tasks", (req, res) => {
```

```
  const { title, description = "", priority = "Medium" } = req.body;
 if (!validateTitle(title)) {    return res.status(400).json({
message: "Title must be between 1 and 50 characters",
    });
  }

  if (!validatePriority(priority)) {     return
res.status(400).json({       message: "Priority must be
Low, Medium, or High",
    });
  }

  const query =
    "INSERT INTO tasks (title,description,priority) VALUES (?,?,?)";

  db.run(
query,
    [title.trim(), description.trim(), priority],
function (error) {
       if (error) {         return res.status(500).json({ message: "Failed to
create task" });
      }

      db.get(
        "SELECT id,title,description,priority,isDone,createdAt FROM tasks WHERE id = ?",
  [this.lastID],
        (err, row) => {
```

```
          if (err) {
            return res.status(500).json({ message: "Task created but fetch failed" });
          }

          res.status(201).json(normalizeTask(row));
        }
      );
    }
  );
});

/* UPDATE TASK */

app.put("/tasks/:id", (req, res) => {

  const { id } = req.params;   const { title, description = "",
priority = "Medium" } = req.body;
   if (!validateTitle(title)) {     return res.status(400).json({
message: "Invalid title" });
  }

  if (!validatePriority(priority)) {   return
  res.status(400).json({ message: "Invalid priority" }); }

  const query =
    "UPDATE tasks SET title=?,description=?,priority=? WHERE id=?";
```

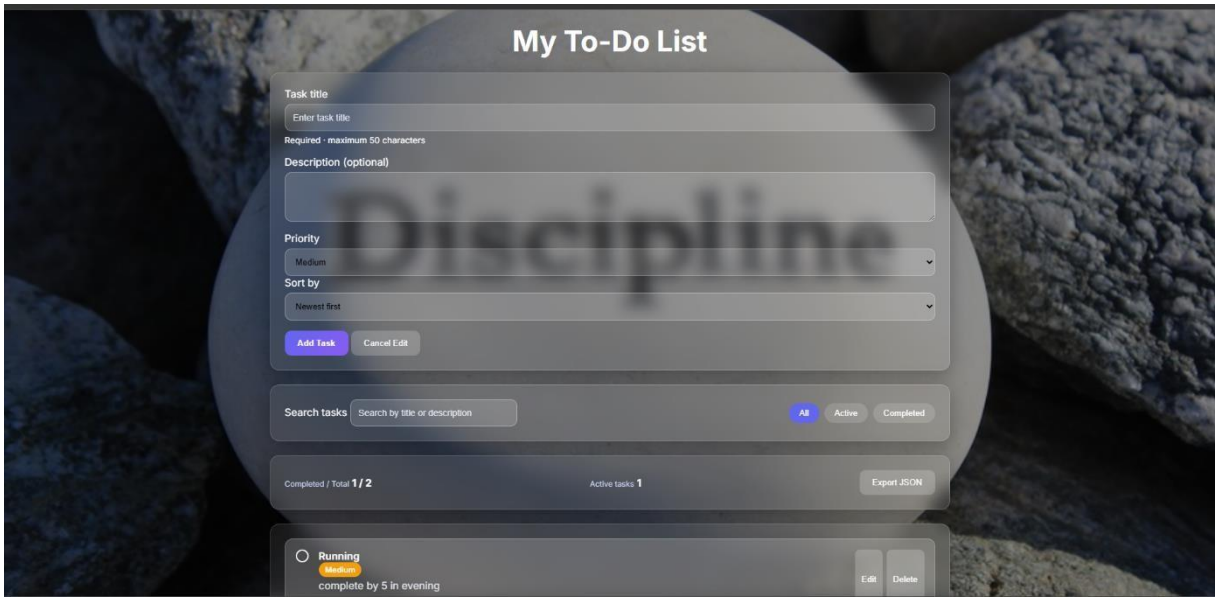
```
  db.run(
query,
    [title.trim(), description.trim(), priority, id],
function (error) {
       if (error) {         return res.status(500).json({ message: "Failed to
update task" });
      }

      if (this.changes === 0) {         return res.status(404).json({
message: "Task not found" });
      }

      db.get(
        "SELECT id,title,description,priority,isDone,createdAt FROM tasks WHERE id=?",
        [id],
        (err, row) => {
         if (err) {
           return res.status(500).json({ message: "Task updated but fetch failed" });
          }
          res.json(normalizeTask(row));
        }
      );
    }
  );
});

/* TOGGLE STATUS */
```

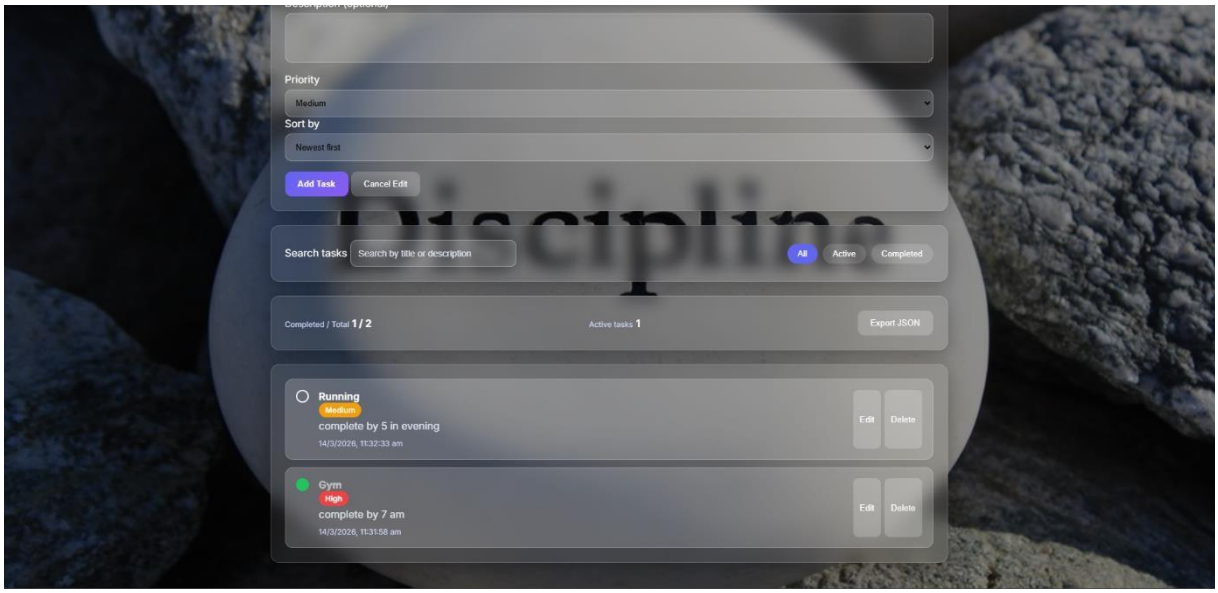
```
app.patch("/tasks/:id/status", (req, res) => {

  const { id } = req.params;
const { isDone } = req.body;

  if (typeof isDone !== "boolean") {     return res.status(400).json({
message: "isDone must be boolean" });
  }

  db.run(
    "UPDATE tasks SET isDone=? WHERE id=?",
    [isDone ? 1 : 0, id],
function (error) {
     if (error) {        return res.status(500).json({ message: "Failed to
 update status" });
      }
      if (this.changes === 0) {     return res.status(404).json({
    message: "Task not found" });
      }

      db.get(
        "SELECT id,title,description,priority,isDone,createdAt FROM tasks WHERE id=?",
        [id],
        (err, row) => {
           if (err)
{
```

```
            return res.status(500).json({ message: "Status updated but fetch failed" });
          }

          res.json(normalizeTask(row));
        }
      );
    }
  );
});

/* DELETE TASK */

app.delete("/tasks/:id", (req, res) => {

 const { id } = req.params;

 db.run(
    "DELETE FROM tasks WHERE id=?",
    [id], function
    (error) {

      if (error) {         return res.status(500).json({ message: "Failed to
delete task" });
      }

      if (this.changes === 0) {         return res.status(404).json({
message: "Task not found" });
```

```
      }

      res.status(204).send();
    }
  );
});

/* FRONTEND ROUTE */

app.get("*", (req, res) => {
res.sendFile(path.join(frontendPath, "index.html"));
});

/* START SERVER */

app.listen(PORT, () => {  console.log(`Server running on
 http://localhost:${PORT}`); });


/* CLOSE DATABASE */

process.on("SIGINT", () => {
db.close(() => {
process.exit(0);
  });
});
```

❖ Output:


My To-Do List
Task title
Enter task title
Required · maximum 50 characters
Description (optional)
Priority
Medium
Sort by
Newest first
Add Task
Cancel Edit
Search tasks
Search by title or description
All
Active
Completed
Completed / Total 1 / 2
Active tasks 1
Export JSON
Running
Medium
complete by 5 in evening
Edit
Delete


Priority
Medium
Sort by
Newest first
Add Task
Cancel Edit
Search tasks
Search by title or description
All
Active
Completed
Completed / Total 1 / 2
Active tasks 1
Export JSON
Running
Medium
complete by 5 in evening
14/3/2026, 11:32:33 am
Edit
Delete
Gym
High
complete by 7 am
14/3/2026, 11:31:58 am
Edit
Delete